Lab 06 — Flannel, CNI & Pod Networking

ID: K8S-LAB-06 · **Category:** Networking · **Level:** Engineer · **Duration:** 60 min

Focus: CNI, Flannel, PodCIDR, overlay, VXLAN

Objective

Understand what CNI actually does, inspect Flannel's pod, routes and config files, and recover from PodCIDR mismatch.

Concepts Covered

• The CNI spec (ADD/DEL/CHECK)

• VXLAN overlay basics

• Per-node /24 subnet from the cluster PodCIDR

Prerequisites

• A cluster from Lab 1 with Flannel

Architecture

Flannel gives every node a /24 from the PodCIDR, plumbs a cni0 bridge, and tunnels off-node traffic via VXLAN over UDP/8472.

```
Pod A (node1, 10.244.1.5)  <-- VXLAN -->  Pod B (node2, 10.244.2.7)
   |
 cni0 bridge
   |
 veth pair  --  Pod netns
```

Lab Steps

01. Inspect Flannel pods

One kube-flannel-ds Pod per node.

```
kubectl -n kube-flannel get pods -o wide
```

Expected: All Running 1/1.

Internals: Flannel runs as a DaemonSet so every node — including new ones added via kubeadm join — automatically gets a Flannel agent.

02. Inspect node subnet allocation

Shows which /24 this node owns.

```
cat /run/flannel/subnet.env
```

Expected: FLANNEL_SUBNET=10.244.0.1/24 (on the control plane).

Internals: Flannel allocates a subnet for the node via the node's spec.podCIDR field.

03. Inspect the CNI config drop

Without this file kubelet refuses to mark Pods Ready.

```
ls /etc/cni/net.d/
```

```
cat /etc/cni/net.d/10-flannel.conflist
```

Expected: JSON with plugins: [flannel, portmap].

Internals: kubelet picks the alphabetically first conflist as the active CNI.

04. Trace pod-to-pod traffic

Proves Pod-to-Pod connectivity across the CNI.

```
kubectl run a --image=busybox --command -- sleep 3600
kubectl run b --image=busybox --command -- sleep 3600
kubectl get pods -o wide
A_IP=$(kubectl get pod a -o jsonpath='{.status.podIP}')
kubectl exec b -- ping -c 3 $A_IP
```

Expected: 3 successful ICMP replies.

Internals: Same node → goes through cni0 bridge. Different nodes → goes through VXLAN tunnel.

05. Simulate a PodCIDR mismatch (don't run on production!)

Demonstrates the failure mode. Then revert.

```
kubectl -n kube-flannel edit configmap kube-flannel-cfg # change Network to 10.99.0.0/16
```

Expected: Flannel pods CrashLoopBackOff with `Network CIDR mismatch`.

Internals: Flannel's agent compares the ConfigMap to the node's podCIDR field. A mismatch is fatal.

Common Errors

Flannel CrashLoopBackOff — Cause: PodCIDR vs ConfigMap mismatch · Fix: Revert ConfigMap, or re-init the cluster with the matching `--pod-network-cidr`

CoreDNS stuck ContainerCreating — Cause: CNI config not written yet · Fix: Wait or `rollout restart` after CNI is healthy

Troubleshooting

• `ip route` on the node should list the per-node subnets via flannel.1

• `bridge link show` shows the cni0 bridge

Enterprise Relevance

Almost every cluster outage that 'feels like networking' is actually CNI. Knowing where /etc/cni/net.d/ and /run/flannel/ live saves hours.

Validation

• Pods can ping each other across nodes

• You can articulate what `subnet.env` is for

Related Incidents & Guide Sections

Incidents: K8S-INC-3003

• Networking & Service types

• CNI failure

Questions

Beginner

Q. What does CNI stand for?

A. Container Network Interface.

Intermediate

Q. Why does every node need its own /24?

A. So Pod IPs are unique and routable without coordination.

Advanced

Q. Why VXLAN over plain IP routing?

A. VXLAN works across L2 boundaries without needing the underlay to learn Pod CIDRs.

Scenario

Q. New node joins, pods on it can't reach pods elsewhere. First check?

A. Flannel pod on the new node, then VXLAN UDP/8472 on the underlay firewall.

Viva

Q. What does the kubelet check before marking a Pod Ready w.r.t. CNI?

A. That CNI ADD succeeded and an IP was returned for the sandbox.